Computing Consistent Levels for Layered Software Architecture Visualizations

Separating Architectural Hypothesis, Package Tree and Local Layout

Johannes Weigend
Weigend AM GmbH & Co.KG, Germany/Brazil
johannes.weigend@weigend.de

Michael Philippsen
Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany
michael.philippsen@fau.de

Keywords: software architecture visualization, layout invariants, layered architecture visualization, strongly connected components, dependency analysis, software engineering tools

Abstract: Layered architecture visualizations communicate dependency structure through vertical position: elements that depend on others sit higher, foundational elements sit lower. Their value rests on a simple promise—an upward edge should reveal an architectural anomaly, not a layout artifact. If level computation is wrong, the drawing still looks plausible while misleading the viewer.

Levelized dependency maps are not new. Structure101's Levelized Structure Map arranges elements into rows and exposes feedback in cyclic regions (Headway Software Technologies Ltd., 2023b; Muccini and Tekinerdogan, 2012; Headway Software Technologies Ltd., 2023a). Published descriptions specify the intended visual behaviour but not the concrete level-computation algorithm or its consistency conditions, leaving it unclear whether an upward edge is a finding, a tolerated cycle cut, or an implementation defect.

In this paper, we present the level pipeline implemented in S202, an open-source Java architecture-visualization tool under Apache 2.0. The pipeline has three stages on three independent graphs. First, S202 detects global SCCs in the class graph and heuristically cuts large cycles. Second, from weighted package dependencies it derives an architectural hypothesis: more heavily used packages sit lower, their users higher. Third, it computes a local level inside each parent container, which determines the vertical placement of siblings.

The algorithmic contribution is the strict separation of architectural hypothesis from layout position. Package analysis decides which package directions are normal, cyclic, or heuristically cut; local placement only decides where classes and visible containers appear inside one concrete parent. This prevents feedback loops in which class feedback, package order, and local positions redefine each other.

Five machine-checkable invariants verify that each run is internally consistent: class dependencies respect class levels after heuristic cuts, dominant weighted package dependencies respect package levels, cyclic peer packages share a level after filtering, visual ranks follow local levels, and cached edge classifications match the current SCC and level state. The checks distinguish pipeline defects from deliberately tolerated heuristic cuts and from architecture violations that the visualization is meant to expose.

We demonstrate the pipeline on Minecraft Forge 1.19.2, a large and highly cyclic system that stresses every stage. Separating global architecture depth from local layout, and preserving the computed package order in the UI instead of reordering it to suppress cycles, reduces the visual upward-edge count from roughly 6 000 to 4 200; the remaining edges are real architecture violations or explicit heuristic cuts. The result is a corrected, reproducible level-computation algorithm whose visible anomalies can be explained and checked.

REFERENCES

Eades, P., Lin, X., and Smyth, W. F. (1993). A fast and effective heuristic for the feedback arc set problem. *Information Processing Letters*, 47(6):319–323.

Headway Software Technologies Ltd. (2023a). Dependency matrix. Structure101 Studio documentation. Available at https://www.sonarsource.com/structure101/docs/java/studio/content/graphs/matrix, accessed 15 May 2026.

Headway Software Technologies Ltd. (2023b). Levelized structure map (lsm). Structure101 Studio documentation. Available at https://www.sonarsource.com/structure101/docs/cpa/studio/Content/restructure101/lsm.html, accessed 15 May 2026.

Karp, R. M. (1972). Reducibility among combinatorial problems. In Miller, R. E. and Thatcher, J. W., editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press.

Muccini, H. and Tekinerdogan, B. (2012). Software architecture tool demonstrations. In *Joint 10th Working Conference on Software Architecture and 6th European Conference on Software Architecture, Companion Volume*, WICSA/ECSA 2012, pages 84–85. ACM.

Appendix A – Why Naive Single-Pass Does Not Work: Failure Modes

The Core Problem: Architectural Hypothesis, Local Layout, Five Consistency Checks

A correct layered layout has to separate an architectural hypothesis from the local placement that makes this hypothesis visible. In S202 this means answering three related questions that are often treated as one ordering problem:

- *Class analysis* – where does a class sit in the global class dependency stack? This is the longest-path depth in the SCC-collapsed class graph.
- *Package analysis* – what package order is implied by the aggregated dependency traffic? This architectural hypothesis is computed on the weighted inter-package graph, independently of the deepest contained class.
- *Local layout* – where should an element's box sit inside its parent container? Only dependencies between classes in the subtrees of that parent's direct children decide this local level.

The three computed values are independent in principle, and the implementation must keep the package-level hypothesis separate from local layout choices. Mixing them is the root cause of every failure mode below. Five consistency checks then inspect the resulting levels and rendered order:

1. **Class-level consistency (R1-algo):** If $A \to B$ denotes a class dependency, then $\text{archLvl}(A) > \text{archLvl}(B)$ after heuristic back-edges are removed. This checks the longest-path assignment in the class-dependency SCC-DAG.
2. **Visual-rank consistency (R1-visual):** For every class dependency $A \to B$ that is not a heuristic back-edge, the rendered position of A in its parent box chain sits above B. The comparison is lexicographic on the tuple of local levels along the parent path, ending with the class's own local level. This is the statement represented by the drawing.
3. **Package-SCC consistency (R2):** Packages that form a cyclic group in the filtered package dependency graph share the same package architecture level.
4. **Package-order consistency (R3):** In the weighted inter-package dependency graph, when the call traffic from package P_A to P_B dominates the reverse, P_A ends up at a strictly higher package architecture level than P_B.
5. **Classification consistency (R5):** Cached edge classifications such as `NORMAL`, `VIOLATION`, and `INTRA_SCC` match the current level assignment and SCC membership.

R1-algo, R2, R3 and R5 check the algorithmic model; R1-visual checks the rendered order. The failure modes that follow show why each obvious shortcut collapses one of these concerns into another and loses information.

Relation to Structure101 and Levelized Structure Maps

Structure101's Levelized Structure Maps are a closely related practical predecessor to the layout studied here. They organize code elements into rows such that, as far as possible, dependencies flow downward, and they expose feedback dependencies when cyclic regions prevent strict levelization (Headway Software Technologies Ltd., 2023b; Headway Software Technologies Ltd., 2023a). This makes Structure101 an important point of comparison and a necessary part of the related work.

The public Structure101 documentation, however, describes the intended properties of the visualization rather than the concrete algorithm that computes the rows. The same is true for academic references that discuss Structure101 as a software architecture analysis tool or as related work for layered visualization (Muccini and Tekinerdogan, 2012): they characterize the LSM concept, but do not provide an algorithmic specification of Structure101's levelization procedure. To the best of our knowledge, no published paper describes the exact Structure101 algorithm.

S202 therefore provides an open algorithmic account of a related class of levelized dependency layouts, not a claim that the visual idiom itself is new. The novelty lies in making the computation explicit, separating class architecture depth, package architecture order, and local layout position, and checking the resulting layout through executable invariants.

Failure Mode 1: Cascading Level Updates

An algorithm iterates over classes and assigns levels. It first sees class A in package P_1, computes $\text{level}(A) = 3$, and therefore sets $\text{level}(P_1) = 3$. Later it reaches class B in package P_2 and discovers that A and B are in the same SCC and must share a level, now $\text{level}(A) = \text{level}(B) = 5$. This forces P_1 to be recomputed. If P_1 depends on P_3, P_3 may need to move as well. A cascade follows.

In a naive single-pass implementation, however, P_1 and P_3 may already have been finalized. SCC membership is only known after the relevant graph has been traversed; if level assignment and SCC discovery are interleaved, the iteration order decides whether the correction is still applied in time.

Failure Mode 2: Interleaved Cycle Detection Produces Classpath-Dependent Results

The most subtle practical failure mode occurs when cycle detection and level computation are not separated. The algorithm traverses the class graph, assigns levels, and handles discovered cycles on the fly, depending on which class happens to be processed first.

Classes come from JARs. The order in which the JVM sees classes follows the order in which they were packaged into the JAR. That order can depend on the build tool, module order in a multi-module build, filesystem ordering on the build server, and other implementation details. None of these orders is semantically meaningful; none encodes architectural dependency direction.

The result is unreliable: the same source code can produce different level layouts under different build configurations. The errors are subtle. The layout may still look plausible, arrows may mostly point in the expected direction, and no obvious visual break appears. Yet the precise hierarchy information is corrupted and may remain unnoticed for a long time.

The solution is to separate the phases strictly: first compute the SCC partition on the complete graph, then build the SCC-DAG, then assign levels. Tarjan's algorithm computes the SCC partition in $O(V+E)$; with deterministic input ordering, the implementation becomes reproducible and independent of classpath traversal artifacts.

Formal Counterexample: Why a Naive Recursive Algorithm Fails

The intuitive level-computation algorithm is a recursive DFS with memoization:

```
level(A):
  if A already computed: return level(A)
  if A has no dependencies: return 0
  return max(level(dep) for dep in A.dependencies) + 1
```

For acyclic graphs this is correct and equivalent to longest-path computation on a DAG. For cyclic graphs it fails structurally.

Counterexample with two SCCs:

```
SCC_1 = {A, B}, with A -> B -> A
SCC_2 = {C, D}, with C -> D -> C
Cross dependency: A -> C
```

The correct levels are $\text{level}(C) = \text{level}(D) = 0$ and $\text{level}(A) = \text{level}(B) = 1$.

Recursive algorithm, starting at A:

```
level(A)  [A "in progress", sentinel = 0]
  -> level(B)
      -> level(A): cycle detected, return 0
    B.level = 0 + 1 = 1
  -> level(C)  [C "in progress", sentinel = 0]
      -> level(D)
          -> level(C): cycle detected, return 0
        D.level = 0 + 1 = 1
    C.level = 1 + 1 = 2   <-- wrong; correct would be 0
  A.level = max(1, 2) + 1 = 3   <-- wrong; correct would be 1
```

After SCC equalization, $\mathrm{SCC}_1 = \max(3,1) = 3$ and $\mathrm{SCC}_2 = \max(2,1) = 2$. Both results are wrong.

The cause is that the recursive algorithm enters SCC_2 from the context of SCC_1. The sentinel-based local cycle handling inflates the level of SCC_2, and that inflated level then feeds into the computation of SCC_1. The correct level of SCC_2, namely zero, is only known after the graph has been collapsed into SCCs and evaluated as an SCC-DAG.

Thus, recursive level algorithms that mix traversal, cycle handling, and level assignment can produce traversal-order-dependent results on graphs with interacting SCCs. In JVM-based tools, that traversal order can be influenced by classpath and packaging order. The correct solution collapses SCCs before level assignment: Tarjan, then SCC-DAG, then longest path.

Failure Mode 3: Deriving Package Levels from Class Levels

An obvious implementation assigns each package a level equal to the maximum level of its contained classes, then lifts parent packages transitively. This is wrong for two reasons.

First, the rule conflates containment with dependency. A package that *contains* a class at level 7 is not necessarily architecturally positioned at level 7 itself; it may have no cross-package dependencies at all. Two disconnected package trees processed from the same JAR would be ordered relative to each other by accident of their class contents, not by their actual dependency relationships.

Second, the approach creates a circular dependency between the two level systems: class levels depend on the class graph, but package levels derived from class levels then constrain the visual position of packages relative to classes, making the combined coordinate system ill-defined.

S202 avoids this by computing class levels and package levels on independent graphs with no mutual constraint. Each system starts at zero and grows according to its own dependency structure.

Failure Mode 4: Large SCCs Without Heuristics Collapse the Layout

Without a mechanism for breaking large SCCs, all members of a cyclic group must be placed on the same level. For small cycles of two to five classes this is acceptable. For Minecraft Forge 1.19.2 (5 123 classes), however, we observed a single SCC containing 2 857 classes. Without a breaker, more than 55 percent of the project would be assigned the same level. All remaining classes would be distributed above, but the internal dependency hierarchy within this large cycle would be invisible.

The heuristic SCC breaker solves this practical problem. It identifies back edges—dependencies that run against the inferred architecture direction—and removes them for level computation. The removed edges are visualized as violations instead of being ignored. The layout preserves visibility of cyclic dependencies while still producing a usable hierarchy.

Relation to Feedback Arc Set Heuristics

Failure Mode 4 is an instance of the Minimum Feedback Arc Set problem: remove a minimal set of directed edges so the graph becomes acyclic. The problem is NP-hard (Karp, 1972), so any practical polynomial-time implementation must use a heuristic or approximation.

A well-known related heuristic is due to Eades, Lin, and Smyth (Eades et al., 1993). It orders nodes according to the difference between outgoing and incoming degree, then classifies edges that point backward in the resulting sequence as feedback edges. The intuition is similar to S202: high out-degree suggests a higher-level element; high in-degree suggests a lower-level element.

S202 uses a deliberately simpler variant: a direct normalized score per node,

$$r(P) = \frac{\mathrm{outWeight}(P) - \mathrm{inWeight}(P)}{\max(1,\ \mathrm{outWeight}(P) + \mathrm{inWeight}(P))},$$

where the weights are summed within the SCC under consideration. At the class level the weights are the in- and out-degrees in the class dependency graph; at the package level they are aggregated method-call counts. An edge $P_A \to P_B$ inside an SCC is classified as a back-edge candidate when $r(P_A) < r(P_B) - 0.1$. Equal-rank pairs represent symmetric bidirectional dependencies with no dominant architectural direction; they remain in the same SCC and are assigned the same level.

The important distinction is transparency. Many tools break cycles implicitly, but S202 classifies heuristic cut edges explicitly. They are marked as `VIOLATION`, rendered in red, and known to the invariant checker. This makes algorithmic defects separable from tolerated heuristic artifacts.

The Solution: Three Computed Values on Independent Graphs

Failure modes 1 to 3 share a root cause: conflating values that belong to different semantic domains. S202 resolves this by computing three independent values, two of them for analysis and one for layout.

- **Class architecture level** is the longest path in the SCC-collapsed class dependency DAG. It represents global architectural depth: how many layers of dependencies separate a class from the bottom of the analysed dependency graph.
- **Package architecture level** is the longest path in the SCC-collapsed weighted inter-package dependency graph. Edge weights are method-call counts aggregated from class-level references. The result represents the module's position in the system's overall dependency order, independently of the class-level coordinate.
- **Local layer index** is the longest path in the SCC-collapsed sibling-only dependency graph of a single parent container. The graph contains only edges whose source and target both lie in some direct child of that parent; references leaving the parent's subtree are excluded because they are handled by the enclosing container. Every element (class and package) gets a local level that is only meaningful within its own parent box. The renderer sorts each parent's children by this index, so the visible Y position of a box is the local level, not the global architecture level.

All three concepts use the same algorithmic shape—Tarjan SCC detection, weight-based back-edge identification, DAG condensation, longest-path propagation—but on three different graphs with three different scopes. Because the systems are independent, packages with no cross-package dependencies correctly land at architecture level 0 regardless of the class levels inside them, and a top-level class like *ArchitectureView* that orchestrates a sub-package such as *ui.rendering* sits visually above the sub-package box without distorting either of their architecture levels.

The separation is an engineering choice driven by semantic clarity. A unified single-coordinate approach is theoretically possible, but it conflates three semantically distinct concerns—global architectural depth, module dependency order, and per-container visual position—into one number, making the pipeline hard to reason about, test, and maintain. Each simplification of such a unified approach turns into one of the failure modes above.

The three concepts have the following algorithmic structure. Phases 1 and 2 compute the global architecture levels; Phase 3 derives the local level per parent container from the architecture-level output.

Algorithm 1: Phase 1 – Global Class Levels (`LevelCalculator`, Step 3)

class dependency graph
→ Tarjan (SCCs)
→ SCCBreaker: identify heuristic back-edges in large SCCs ($|S| \geq 3$; 2-class SCCs are equalized instead)
$\text{rank}(P) = (\text{outDeg} - \text{inDeg}) \,/\, \max(1, \text{outDeg} + \text{inDeg})$
edge $A \to B$ is a back-edge candidate if $\text{rank}(A) < \text{rank}(B) - 0.1$
→ Remove back-edges → filtered class DAG
→ Condense into SCC-DAG → longest-path pass
→ Assign SCC level to all member classes (equalization implicit)

Algorithm 2: Phase 2 – Package Architecture Levels (`LevelCalculator`, Step 4)

Build weighted inter-package dependency graph $G_P = (V_P, E_P, w)$:
For each class C in package P_{leaf} with method calls to class D in P_{target} ($P_{\text{leaf}} \neq P_{\text{target}}$):
$w(P_{\text{ancestor}}, P_{\text{target}})$ += callCount$(C \to D)$
for every ancestor P_{ancestor} of P_{leaf} with $P_{\text{ancestor}} \neq P_{\text{target}}$
Fallback weight = 1 for structural references with no method-call data
No directional filtering: both parent→child and child→ancestor edges enter G_P. The SCC breaker decides direction by weight.

Iterative SCC-breaking (repeat until stable):
Tarjan on current graph → find SCCs with $|S| \geq 2$ (2-package SCCs broken if rank differs; equalized otherwise)

For each member $P \in S$:
$r(P) = \left(\sum_{Q \in S} w(P,Q) - \sum_{Q \in S} w(Q,P)\right) \,/\, \max\left(1, \sum_{Q \in S} w(P,Q) + \sum_{Q \in S} w(Q,P)\right)$
Edge $P_A \to P_B$ inside S is a back-edge if $r(P_A) < r(P_B) - 0.1$; remove it
Equal-rank pairs remain in the SCC → assigned the same level

Level assignment:
Condense remaining graph into SCC-DAG
Longest-path propagation on SCC-DAG
Assign SCC level to all member packages

Phase 1 produces classpath-independent class architecture levels because SCCs are collapsed before levels are assigned. Phase 2 produces package architecture levels directly from package dependencies: every class reference contributes to the weighted graph regardless of the containment relation between source and target package, and a per-SCC rank score decides which direction of a cyclic group dominates. The two architecture coordinate systems do not constrain each other—a package with no outgoing cross-package calls lands at level 0 regardless of the class levels inside it, and conversely the class levels of a package's contents do not lift the package itself.

Algorithm 3: Phase 3 – Local Level per Parent Container (`LocalLevelCalculator`, Step 6)

for each parent package P with at least two direct children:
Let $S(P)$ be the direct children (classes and sub-packages).
Build sibling-only weighted graph G_P^{loc} on $S(P)$:
edge $X \to Y$ with weight $\sum$callCount$(c \to d)$
over all class refs $c \to d$ where c lies in the subtree of sibling X
and d lies in the subtree of sibling Y;
refs leaving the parent's subtree are excluded.
Tarjan on $G_P^{\text{loc}} \to$ SCCs.
Within each SCC of size ≥ 2: evaluate every candidate edge cut.
Remove the edge whose removal maximally decomposes the SCC;
use package-architecture contradiction, edge weight, and rank only as tie-breakers.
Condense to SCC-DAG, longest-path pass, assign **localLevel** per sibling.

Phase 3 is a pure layout phase. It never modifies architecture levels. Each parent container gets its own computation: a class's local level inside *ui* carries no relationship to a class's local level inside *ui.rendering*. The renderer sorts each parent's children by their local layer index, descending. The enclosing parent's local level handles cross-container ordering recursively. The local SCC breaker resolves local cycles—for example, two sibling sub-packages with mutual class references—silently. It removes one edge at a time, preferring the cut that reduces the remaining cyclic structure the most; package architecture, edge weight and rank only decide among otherwise equivalent candidates. S202 does not surface these local cycles as a separate Tangle category because the user-visible tangle list remains a global architecture statement.

	Phase 1 – Class	Phase 2 – Package	Phase 3 – Local Level
Graph	Class dependency	Weighted inter-package	Sibling-only weighted (per parent)
Edge weight	Unweighted	Method-call counts	Method-call counts
Scope	Global	Global	Per parent container
Meaning	Class arch. depth	Module dep. position	Visual Y in parent box
Zero point	Class with no deps	Package with no outgoing calls	Bottom sibling in box
Cycles	SCCBreaker (degree)	Weight-rank, all edges	Max-cycle-break within siblings
Used by	R1-algo, Tangles	R3, R2	R1-visual, renderer sort

Table 1: The three independent values computed by S202.

Layout Invariants for Level Correctness

The three level phases are guarded by five machine-checkable consistency checks. The `LayoutInvariantChecker` checks R1-algo, R2, R3, and R5 as algorithm consistency checks after every analysis run; the architecture builder checks R1-visual through violation detection on the visual-rank tuple. These checks are evidence that the computed model is internally consistent; they are not the primary algorithmic contribution.

Rule	Consistency check
R1-algo	Non-heuristic class dependencies respect class architecture levels: $A \to B \Rightarrow \text{archLvl}(A) > \text{archLvl}(B)$. Tests the longest-path assignment of Phase 1.
R1-visual	For every non-heuristic class dependency $A \to B$, the rendered position of A sits above B. The check compares positions lexicographically on $\left[L(\text{ancestor}_1), \ldots, L(\text{ancestor}_n), L(A)\right]$ versus $\left[L(\text{ancestor}'_1), \ldots, L(\text{ancestor}'_m), L(B)\right]$ where L denotes the local level. Covers same-parent, same-grandparent, and cross-tree comparisons uniformly.
R2	Packages forming a cyclic group in the filtered package dependency graph share the same package architecture level.
R3	For every weighted package edge (P_A, P_B) with $w(P_A, P_B) > w(P_B, P_A)$ that is not a back-edge cut by the SCC breaker: $\text{archLvl}(P_A) > \text{archLvl}(P_B)$.
R5	Cached edge classifications (`NORMAL`, `VIOLATION`, `INTRA_SCC`) remain consistent with the current level assignment and SCC membership.

Table 2: Five layout consistency checks verified after every analysis run. R1-algo, R3 and R2 check the algorithm; R1-visual checks the rendered picture; R5 keeps the cached annotations consistent.

Throughout the iterative development of the level pipeline, R2 surfaced implementation bugs after algorithm changes that the unit-test suite had not detected. Figure 1 shows a representative R2 report. In *software-ekg-7*, for instance, an R2 finding exposed a missing package-SCC equalization step and directly triggered a pipeline fix.

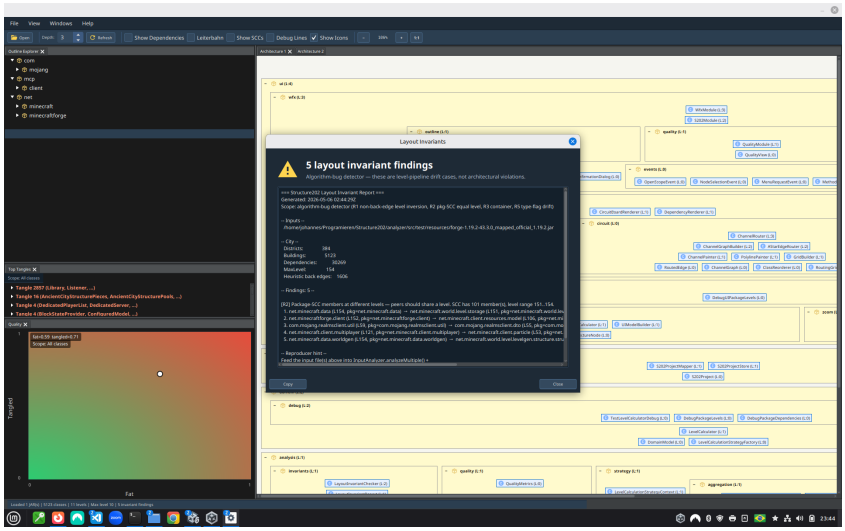


Figure 1: R2 checker report: packages in a filtered package-SCC occupy different levels. The checker reports findings as algorithm-bug candidates with a reproducer block, which makes the visual failure traceable to concrete level-pipeline consistency conditions.

R2 also guided refinement of the rule itself. Against Minecraft Forge (2 895 classes, 220 packages, 188 of 207 packages in a single package-level SCC), two R2 reports turned out to be *false positives*: R2 filtered class-level heuristic back-edges but not the package-level back-edges that the package SCC-breaker had already removed. R3 already applied the correct filter; once R2 was aligned, the Minecraft findings disappeared and all 155 invariant tests pass. The invariant checker connects the visual symptom to a reproducible algorithm defect.

The Remaining Rules: R1-algo, R1-visual, R3, and R5

R1-algo ($A \to B \Rightarrow \text{archLvl}(A) > \text{archLvl}(B)$ on the filtered class graph) is the most direct invariant on the analysis side: it is a corollary of longest-path computation on the SCC-DAG and holds trivially for any correctly implemented class-level assignment. Its value lies in providing an immediate check—a failing R1-algo signals a fundamental breakdown in the class-level pipeline.

R1-visual (lexicographic comparison of local-layer-index tuples along the parent chain) is the invariant that the drawing represents. We separate the two checks intentionally: R1-algo asks "did Phase 1 compute consistent class architecture levels?", R1-visual asks "does the rendered Y position reflect the class dependencies?" Both pass on a correct implementation; only the second one may fail when the sibling-only graph of some parent exposes a genuine upward edge, because that is a real architecture violation rather than an algorithm bug. The single visual-rank comparison covers every pair: siblings inside the same container compare on their final tuple element; cross-tree pairs diverge earlier; classes versus sub-package boxes in the same parent compare on the shared parent context. The comparison needs no special-case logic for parent→child versus sibling versus cross-package class references.

R3 verifies that the dominant direction in the weighted package dependency graph appears in the architecture level assignment. For any package edge (P_A, P_B) where $w(P_A, P_B) > w(P_B, P_A)$—meaning P_A calls into P_B significantly more than vice versa—the invariant requires $\text{archLvl}(P_A) > \text{archLvl}(P_B)$. R3 excludes package back-edges identified during SCC-breaking from this check, analogously to how R1-algo excludes class-level heuristic back-edges. A failing R3 indicates that the weight-based SCC-breaking cut the wrong package edge, or that the package-level DAG propagation did not converge correctly.

R5 validates that every edge's stored classification (`NORMAL`, `VIOLATION`, `INTRA_SCC`) is still consistent with the current level assignment and SCC membership. The analysis computes edge labels and caches them in the model; after any algorithmic change they can silently become stale. R5 is therefore a cross-cutting consistency check: it ties the structural output of the level pipeline back to the semantic annotations visible to the user.

R4: A Retired Rule and What It Teaches

An early version of the invariant checker contained R4:

`Cyclic child packages must share a level.`

The rule sounds reasonable and is correct in simple cases. In practice it failed because the raw package graph contains relationships that should not be interpreted as peer-level architectural cycles:

- **Parent-child edges** create apparent bidirectional relationships because a parent contains its children and children belong to their parent. Without filtering, many parent/child pairs look cyclic.
- **SCCBreaker back edges** connect packages that are not necessarily architectural peers. A rule that includes these edges equalizes the wrong candidates.
- **Shared class SCCs** can make two packages appear package-dependent without representing a true structural peer relationship.

R4 was replaced by R2, which expresses the same intention on the correctly filtered graph. R2 removes class-level heuristic back-edges, package-level SCC-breaking back-edges, intra-subtree edges, and edges caused by shared class SCCs before running Tarjan on the remaining package graph.

Specifying invariants for a three-value level pipeline turns out to be harder than it appears. The checker therefore provides a second safety net: it not only tests the implementation against the rules, but also exposes when the rules themselves are too broad or too coarse. False positives force the rule to become more precise. The later R2 finding in *software-ekg-7* shows that the refined rule was precise enough to distinguish real pipeline bugs from apparent ones.

Implementation: The Complete Pipeline

The conceptual structure expands into the following concrete implementation steps, which constitute the reference implementation (Appendix B).

Algorithm 1: Complete Level Pipeline (`LevelCalculator`)

Step 1: Create class objects (all levels $\leftarrow 0$)
Step 2: Create package objects (all levels $\leftarrow 0$)

Step 3: Compute class architecture levels (Phase 1)
$\rightarrow$ Tarjan on class dependency graph
$\rightarrow$ SCCBreaker: identify heuristic back-edges in large SCCs ($|S| \geq 3$):
$r(C) = (\text{outDeg} - \text{inDeg}) \,/\, \max(1, \text{outDeg} + \text{inDeg})$;
edge $A \rightarrow B$ is a back-edge candidate if $r(A) < r(B) - 0.1$
$\rightarrow$ Remove identified back-edges $\rightarrow$ filtered class DAG
$\rightarrow$ Condense into SCC super-nodes
$\rightarrow$ Single longest-path pass on condensed DAG
$\rightarrow$ Assign SCC level to all member classes

Step 4: Compute package architecture levels (Phase 2)
4a. Build weighted inter-package dependency graph:
$w(P_A, P_B) = \sum$ method-call counts from subtree(P_A) to P_B;
propagate weights up to all ancestor packages (no directional filtering);
both parent$\rightarrow$child and child$\rightarrow$ancestor edges enter the graph

4b. Iterative package SCC-breaking (repeat until stable):
Tarjan on current graph $\rightarrow$ SCCs with $|S| \geq 2$;
$r(P) = (\sum_{Q \in S} w(P,Q) - \sum_{Q \in S} w(Q,P)) \,/\, \max(1, \ldots)$;
edge $P_A \rightarrow P_B$ is a back-edge if $r(P_A) < r(P_B) - 0.1$; remove it;
equal-rank pairs remain in the same SCC $\rightarrow$ assigned the same level

4c. Level assignment on condensed package SCC-DAG:
longest-path propagation; assign SCC level to all member packages.
No child–parent lift: layout positioning is based on `localLevel`.

Step 5: Set reverse dependencies (dependents)

Step 6: Compute local level per parent (Phase 3)
for each parent package P with two or more direct children:
build sibling-only weighted graph G_P^{loc} from class refs
whose source and target both live in some direct child of P;
Tarjan $\rightarrow$ rank-based SCC break $\rightarrow$ longest-path;
assign `localLevel` to each direct child of P.

After Step 3, a class's architecture level is the level of the SCC node to which it belongs, measured as the longest path in the cycle-free quotient graph across all classes of the analysed project. Cycles do not influence level assignment internally because they have been collapsed before longest-path computation begins.

Step 4 treats class architecture levels and package architecture levels as independent coordinate systems. A package with no outgoing cross-package calls receives architecture level 0, regardless of the class levels inside it; two package trees with no shared dependencies are each levelled from zero in their own connected component of the weighted package graph. The earlier child–parent lift, which inflated parent package levels by the class levels their children called, is gone – it was a workaround for the missing layout coordinate.

Step 6 introduces the layout coordinate. Each parent container is treated as a self-contained scope: only class references that stay within the parent's subtree contribute to the local sibling graph. The result is that the box for *ArchitectureView* in the *ui* package sorts above the box for the *ui.rendering* sub-package, simply because *ArchitectureView* has heavier outgoing call traffic into rendering than rendering has back into *ui*. None of this disturbs Phase 1 or Phase 2.

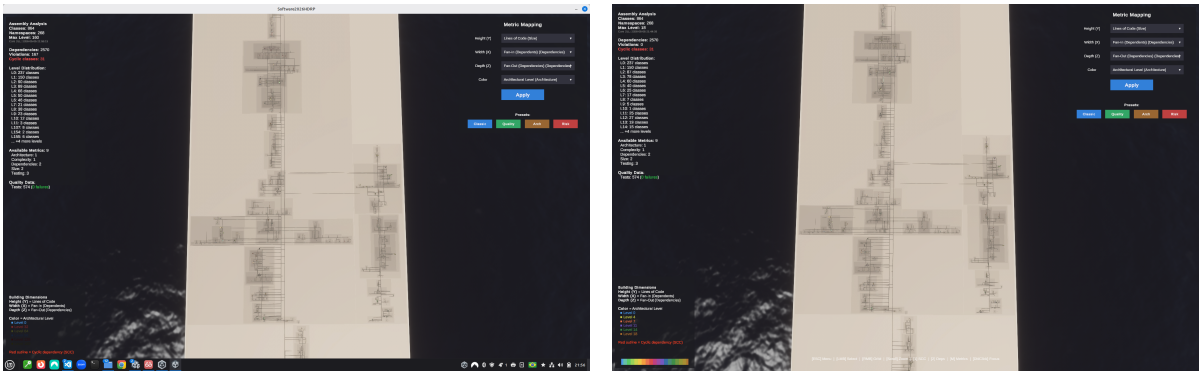
Algorithm 2: Phase 4 – Horizontal Row Ordering (`HorizontalRowLayoutOptimizer`)

for each package *P*:
  Step 1: Group direct children by their `localLevel` value
  Step 2: Collect subtree classes per row element
  Step 3: Build weighted undirected proximity links from inter-sibling class dependencies
  Step 4: Run bounded barycentric sweeps across rows
  Step 5: Store the result in `horizontalLayoutOrder`
rendering: sort a copied child view by `localLevel` (Phase 3), then `horizontalLayoutOrder` (Phase 4), then full name

Phase 4 does not change any level or layer assignment and is deliberately excluded from the layout invariants. Its only purpose is cosmetic: when an overlay of dependency arrows is displayed, same-row elements that share many inter-sibling calls are placed horizontally near each other so the arrows are shorter and less crossed. The implementation stores this as a separate `horizontalLayoutOrder` field and never mutates the semantic child list.

R2 Fix in *software-ekg-7*: Visual Evidence

The *software-ekg-7* case is the concrete failure that motivated the package-level pipeline. Before the fix, the visualization still looked plausible: the package rectangles and dependency edges formed a readable layout, and no single visual element made the pipeline defect obvious. The metrics panel, however, exposed the inconsistency: 167 dependency violations were reported and the maximum architectural level had inflated to 160. After the package-level equalization was corrected, the same input produced zero violations and a maximum level of 18. The number of classes, packages, dependencies, cyclic classes, and tests stayed constant; the change is therefore attributable to level propagation, not to a different input model.



(a) Before the fix: the full visualization looks plausible, but the metric panel reports 167 violations and MaxLevel 160.

(b) After the fix: the same analyzed system reports zero violations and MaxLevel 18.

Figure 2: Full-screen before/after screenshots of the *software-ekg-7* visualization. The visible structure changes only subtly; the level metrics reveal the correction.

(a) Before: 167 violations, MaxLevel 160.

(b) After: zero violations, MaxLevel 18.

Figure 3: Cropped metric panels from Figure 2. These panels are the clearest user-visible symptom of the propagation defect and its fix.

Summary

Correct level computation for layered architecture visualizations requires keeping three questions separate: where does a class sit in the global class dependency stack, where does a package sit in the weighted package dependency order, and where should an element's box go inside its parent container? Every straightforward single-pass approach over a unified graph collapses these questions into one coordinate, and the failure modes documented above show why each obvious simplification breaks down in practice.

S202 resolves this by computing three values on three independent graphs. *Class architecture levels* are the longest path in the SCC-collapsed class dependency DAG; they represent global architectural depth. *Package architecture levels* derive from a weighted inter-package dependency graph whose edges carry aggregated method-call counts; they represent module dependency position independently of class levels. *Local layer indices* come from the per-parent sibling-only graph closed within that parent's subtree; they decide visual Y position inside each box without disturbing either architecture coordinate. All three use the same algorithmic shape: Tarjan SCC detection, weight-based back-edge identification, DAG condensation, longest-path propagation.

A horizontal row-ordering pass (Phase 4) improves the legibility of dependency overlays without modifying any computed value or affecting any layout invariant. Five machine-checkable consistency checks (R1-algo, R1-visual, R2, R3, R5) run after each analysis and emit reproducer blocks for each finding. They provide regression protection and diagnostic evidence; the algorithmic contribution is the separation of the three values themselves.

Two cases demonstrate the value of the separation. In *software-ekg-7*, a plausible layered layout reported 167 dependency violations and a maximum architectural level of 160 before a package-level equalization fix; after the fix, the same input produced zero violations and a maximum level of 18 with no change to the analysed source code. On Minecraft Forge 1.19.2, introducing the local level as a distinct layout coordinate, lifting the parent filter from the weighted package graph, and preserving the computed package order in the UI instead of reordering it to suppress cycles, reduced the visual upward-edge count from roughly 6 000 to 4 200; the eliminated edges were artifacts of forcing parent packages below their sub-packages to keep them out of the algorithm's tangle list, and the 4 200 remaining edges are real architecture violations or explicit heuristic cuts that deserve a code-level fix.

Appendix B – LevelCalculator Reference Implementation

The following source excerpt is the complete Java implementation of the level pipeline discussed above.

```java
package de.weigend.s202.domain.architecture;

import de.weigend.s202.domain.DomainModel;
import de.weigend.s202.domain.strategy.LevelCalculationStrategyContext;
import de.weigend.s202.graph.StronglyConnectedComponent;
import de.weigend.s202.graph.TarjanSCCFinder;
import de.weigend.s202.reader.DependencyModel;

import java.util.*;
import java.util.logging.Logger;

/**
 * Calculates architectural levels for classes and packages based on dependencies.
 *
 * Pipeline:
 *   Step 1  Create class objects       (all levels = 0)
 *   Step 2  Create package objects     (all levels = 0)
 *   Step 3  Compute class levels       strategy (Tarjan → SCC-break → DAG → longest-path)
 *   Step 4  Compute package levels     weighted inter-package graph → SCC-break → DAG → longest-path
 *   Step 5  Set reverse dependencies
 *   Step 6  Assign local layer index   per-parent sibling graph → SCC-break → DAG → longest-path
 *                                      (rendering position within each parent box, no global meaning)
 *
 * Package levels are computed independently from class levels using a weighted
 * inter-package dependency graph. The weight of an edge P_A → P_B is the
 * aggregated method-call count from classes in P_A's subtree to classes in P_B,
 * with a fallback weight of 1 for structural references without method-call data.
 * This separates the containment hierarchy from the dependency hierarchy and
 * correctly handles disconnected package trees (they are levelled independently,
 * both starting at 0).
 *
 * Package SCC-breaking uses a weight-based rank score:
 *   rank(P) = (Σ outgoing weights within SCC − Σ incoming weights within SCC)
 *             / max(1, sum of both)
 * Edge P_A→P_B is a back-edge when rank(P_A) < rank(P_B) − 0.1.
 * Equal-rank pairs (symmetric dependency) are left in the same SCC and assigned
 * the same level -- they are genuine peers with no dominant dependency direction.
 */
public class LevelCalculator {

    private static final Logger LOG = Logger.getLogger(LevelCalculator.class.getName());
    private static final double RANK_THRESHOLD = 0.1;

    private final LevelCalculationStrategyContext strategyContext;

    public LevelCalculator() {
        this(LevelCalculationStrategyFactory.createDefault());
    }

    public LevelCalculator(LevelCalculationStrategyContext strategyContext) {
        Objects.requireNonNull(strategyContext, "strategyContext cannot be null");
        this.strategyContext = strategyContext;
    }

    public DomainModel calculate(DependencyModel rawModel) {
        DomainModel model = new DomainModel();

        // Step 1: Create class objects (all levels = 0)
        for (String className : rawModel.getAllClassNames()) {
```

```
60              DependencyModel.ClassInfo rawClass = rawModel.getClass(className);
61              model.addClass(className, new DomainModel.CalculatedElementInfo(
62                  className, rawClass.simpleName, "CLASS", 0,
63                  new HashSet<>(rawClass.dependencies), rawClass.interfaceType));
64          }
65
66          // Step 2: Create package objects (all levels = 0)
67          for (String packageName : rawModel.getAllPackageNames()) {
68              DependencyModel.PackageInfo rawPkg = rawModel.getPackage(packageName);
69              model.addPackage(packageName, new DomainModel.CalculatedElementInfo(
70                  packageName, rawPkg.simpleName, "PACKAGE", 0, new HashSet<>()));
71          }
72
73          // Step 3: Compute class levels
74          calculateClassLevels(model);
75
76          // Step 4: Compute package levels from weighted inter-package graph.
77          calculatePackageLevels(model, rawModel);
78
79          // Step 5: Set reverse dependencies
80          updateDependentRelationships(model);
81
82          // Step 6: Assign per-parent local layer index -- independent of the
83          // global architectureLevel, used by the renderer to position
84          // siblings within each parent's box.
85          new LocalLevelCalculator().assign(model, rawModel);
86
87          return model;
88      }
89
90      public LevelCalculationStrategyContext getStrategyContext() {
91          return strategyContext;
92      }
93
94      // ------------------------------------------------------------------------
95      // Step 3 -- class levels
96      // ------------------------------------------------------------------------
97
98      private void calculateClassLevels(DomainModel model) {
99          Map<String, Set<String>> classDeps = new HashMap<>();
100         for (DomainModel.CalculatedElementInfo c : model.getAllClasses().values()) {
101             classDeps.put(c.fullName, new HashSet<>(c.dependencies));
102         }
103         Map<String, Integer> levels =
104             strategyContext.getClassLevelStrategy().calculateClassLevels(classDeps);
105         for (Map.Entry<String, Integer> e : levels.entrySet()) {
106             DomainModel.CalculatedElementInfo info = model.getClass(e.getKey());
107             if (info != null) info.setArchitectureLevel(e.getValue());
108         }
109     }
110
111     // ------------------------------------------------------------------------
112     // Step 4 -- package levels via weighted inter-package graph
113     // ------------------------------------------------------------------------
114
115     private void calculatePackageLevels(DomainModel model, DependencyModel rawModel) {
116         if (model.getAllPackages().isEmpty()) return;
117         Set<String> allPkgNames = model.getAllPackages().keySet();
118
119         // Build weighted graph: weight[from][to] = aggregated method-call
120         // count from classes in 'from' to classes in 'to', with fallback
121         // weight 1 for structural dependencies without method-call data.
```

```
122         Map<String, Map<String, Integer>> weights = buildWeightedPackageGraph(model, rawModel);
123
124         // Unweighted adjacency for Tarjan (direction matters, zero-weight edges excluded)
125         Map<String, Set<String>> graph = new LinkedHashMap<>();
126         for (String pkg : allPkgNames) graph.put(pkg, new LinkedHashSet<>());
127         for (Map.Entry<String, Map<String, Integer>> entry : weights.entrySet()) {
128             graph.get(entry.getKey()).addAll(entry.getValue().keySet());
129         }
130
131         // Iteratively break SCCs using weight-based rank, then assign levels
132         // via SCC-collapsed DAG longest-path. Remaining SCCs (equal-rank peers)
133         // are collapsed to the same level without cutting any edge.
134         Set<String> backEdgeKeys = new LinkedHashSet<>(); // "from\0to" -- tracked for R3
135         boolean changed = true;
136         while (changed) {
137             changed = false;
138             for (StronglyConnectedComponent scc : new TarjanSCCFinder(graph).findSCCs()) {
139                 if (scc.getSize() < 2) continue;
140                 Set<String> members = scc.getMembers();
141
142                 // Compute weight-based rank for each member within this SCC
143                 Map<String, Double> rank = new HashMap<>();
144                 for (String m : members) {
145                     int out = 0, in = 0;
146                     for (String other : members) {
147                         if (other.equals(m)) continue;
148                         out += weights.getOrDefault(m, Map.of()).getOrDefault(other, 0);
149                         in  += weights.getOrDefault(other, Map.of()).getOrDefault(m, 0);
150                     }
151                     rank.put(m, (out - in) / (double) Math.max(1, out + in));
152                 }
153
154                 // Identify back-edges: from→to where rank(from) < rank(to) - threshold
155                 for (String from : new ArrayList<>(members)) {
156                     for (String to : new ArrayList<>(graph.getOrDefault(from, Set.of()))) {
157                         if (!members.contains(to)) continue;
158                         if (rank.get(from) < rank.get(to) - RANK_THRESHOLD) {
159                             String key = from + "\0" + to;
160                             if (backEdgeKeys.add(key)) {
161                                 graph.get(from).remove(to);
162                                 changed = true;
163                             }
164                         }
165                     }
166                 }
167             }
168         }
169
170         // Assign package levels: SCC-collapsed DAG → longest-path
171         List<StronglyConnectedComponent> sccs = new TarjanSCCFinder(graph).findSCCs();
172         Map<String, StronglyConnectedComponent> pkgToScc = new HashMap<>();
173         for (StronglyConnectedComponent scc : sccs) {
174             for (String m : scc.getMembers()) pkgToScc.put(m, scc);
175         }
176
177         // SCC dependency graph (between SCC IDs)
178         Map<Integer, Set<Integer>> sccDeps = new HashMap<>();
179         for (StronglyConnectedComponent scc : sccs) {
180             sccDeps.put(scc.getId(), new HashSet<>());
181             for (String m : scc.getMembers()) {
182                 for (String to : graph.getOrDefault(m, Set.of())) {
183                     StronglyConnectedComponent toScc = pkgToScc.get(to);
```

```
                    if (toScc != null && toScc.getId() != scc.getId()) {
                        sccDeps.get(scc.getId()).add(toScc.getId());
                    }
                }
            }
        }

        // Longest-path levels on the SCC-collapsed DAG. The previous
        // "childPkgUsedLevel" lift (which inflated a parent's level by
        // the class levels its child packages used) is gone -- layout
        // positioning lives on localLevel now, so architectureLevel
        // can be a direct dependency-chain depth.
        Map<Integer, Integer> sccLevels = new HashMap<>();
        for (StronglyConnectedComponent scc : sccs) sccLevels.put(scc.getId(), 0);
        boolean lvlChanged = true;
        while (lvlChanged) {
            lvlChanged = false;
            for (StronglyConnectedComponent scc : sccs) {
                int maxDep = -1;
                for (int depId : sccDeps.getOrDefault(scc.getId(), Set.of())) {
                    maxDep = Math.max(maxDep, sccLevels.getOrDefault(depId, 0));
                }
                int newLevel = maxDep >= 0 ? maxDep + 1 : 0;
                if (sccLevels.get(scc.getId()) != newLevel) {
                    sccLevels.put(scc.getId(), newLevel);
                    lvlChanged = true;
                }
            }
        }

        // Apply levels to all packages
        Map<String, DomainModel.CalculatedElementInfo> pkgInfos = model.getAllPackages();
        for (StronglyConnectedComponent scc : sccs) {
            int level = sccLevels.get(scc.getId());
            for (String member : scc.getMembers()) {
                DomainModel.CalculatedElementInfo pkg = pkgInfos.get(member);
                if (pkg != null) pkg.setArchitectureLevel(level);
            }
        }

        // Store the weighted graph and identified back-edges in the model
        model.setPackageEdgeWeights(weights);
        model.setPackageBackEdges(backEdgeKeys);

        // Populate package dependencies (unweighted) for reverse-dependency tracking
        for (Map.Entry<String, Map<String, Integer>> entry : weights.entrySet()) {
            DomainModel.CalculatedElementInfo pkg = pkgInfos.get(entry.getKey());
            if (pkg != null) entry.getValue().keySet().forEach(pkg::addDependency);
        }
    }

    /**
     * Builds the weighted inter-package dependency graph.
     * weight(P_A → P_B) = total method-call count from classes in P_A's subtree
     * to classes in P_B. Method calls are a stronger signal than distinct-class
     * counts: a class called hundreds of times clearly dominates one called once,
     * making SCC-breaking direction unambiguous. Intra-subtree calls are excluded.
     * Child-to-ancestor and parent-to-child edges are included.
     * Each call count is propagated to all ancestor packages up to the point where
     * the ancestor would enter the same subtree as the target.
     */
    private Map<String, Map<String, Integer>> buildWeightedPackageGraph(
```

```
246             DomainModel model, DependencyModel rawModel) {
247         Map<String, Map<String, Integer>> weights = new HashMap<>();
248         for (String pkg : model.getAllPackages().keySet()) weights.put(pkg, new LinkedHashMap<>());
249 
250         Set<String> allPkgNames = weights.keySet();
251 
252         for (DomainModel.CalculatedElementInfo cls : model.getAllClasses().values()) {
253             String leafPkg = extractPackageName(cls.fullName);
254             if (leafPkg == null || !allPkgNames.contains(leafPkg)) continue;
255 
256             // Count method calls per target package using raw model data
257             Map<String, Integer> callCountPerPkg = new LinkedHashMap<>();
258             DependencyModel.ClassInfo rawCls = rawModel.getClass(cls.fullName);
259             if (rawCls != null) {
260                 for (DependencyModel.MethodInfo method : rawCls.methods.values()) {
261                     for (Map.Entry<String, Integer> call : method.methodCalls.entrySet()) {
262                         // Method call key format: "ownerClass.methodName"
263                         // Find the target class by checking known dependencies
264                         String calledKey = call.getKey();
265                         for (String dep : cls.dependencies) {
266                             if (calledKey.startsWith(dep + ".")) {
267                                 String toPkg = extractPackageName(dep);
268                                 if (toPkg != null && !leafPkg.equals(toPkg)
269                                         && allPkgNames.contains(toPkg)) {
270                                     callCountPerPkg.merge(toPkg, call.getValue(), Integer::sum);
271                                 }
272                                 break;
273                             }
274                         }
275                     }
276                 }
277             }
278             // Fall back to 1 per target package for structural deps with no call data
279             for (String dep : cls.dependencies) {
280                 String toPkg = extractPackageName(dep);
281                 if (toPkg != null && !leafPkg.equals(toPkg) && allPkgNames.contains(toPkg)) {
282                     callCountPerPkg.putIfAbsent(toPkg, 1);
283                 }
284             }
285 
286             // Propagate each weight up to ancestor packages. Parent->child
287             // edges are no longer filtered here -- the rank-based SCC breaker
288             // decides direction based on actual call-count weight. Layout
289             // positioning is the LocalLevelCalculator's job (step 6), so the
290             // global package level can be a direct dependency-chain count.
291             for (Map.Entry<String, Integer> entry : callCountPerPkg.entrySet()) {
292                 String toPkg = entry.getKey();
293                 int callCount = entry.getValue();
294                 String ancestor = leafPkg;
295                 while (ancestor != null && allPkgNames.contains(ancestor)) {
296                     if (ancestor.equals(toPkg)) break;
297                     weights.get(ancestor).merge(toPkg, callCount, Integer::sum);
298                     ancestor = extractPackageName(ancestor);
299                 }
300             }
301         }
302         return weights;
303     }
304 
305     // ------------------------------------------------------------------------
306     // Step 5 -- reverse dependencies
307     // ------------------------------------------------------------------------
```

```
308
309     private void updateDependentRelationships(DomainModel model) {
310         for (DomainModel.CalculatedElementInfo cls : model.getAllClasses().values()) {
311             for (String dep : cls.dependencies) {
312                 DomainModel.CalculatedElementInfo d = model.getClass(dep);
313                 if (d != null) d.addDependent(cls.fullName);
314             }
315         }
316         for (DomainModel.CalculatedElementInfo pkg : model.getAllPackages().values()) {
317             for (String dep : pkg.dependencies) {
318                 DomainModel.CalculatedElementInfo d = model.getPackage(dep);
319                 if (d != null) d.addDependent(pkg.fullName);
320             }
321         }
322     }
323
324     // ------------------------------------------------------------------------
325     // Utilities
326     // ------------------------------------------------------------------------
327
328     private static boolean isInSameSubtree(String a, String b) {
329         return a.startsWith(b + ".") || b.startsWith(a + ".");
330     }
331
332     private static String extractPackageName(String className) {
333         if (className == null || !className.contains(".")) return null;
334         return className.substring(0, className.lastIndexOf('.'));
335     }
336 }
```

The companion class `LocalLevelCalculator` implements Phase 3 (Step 6).

```
1  package de.weigend.s202.domain.architecture;
2
3  import de.weigend.s202.domain.DomainModel;
4  import de.weigend.s202.domain.DomainModel.CalculatedElementInfo;
5  import de.weigend.s202.graph.StronglyConnectedComponent;
6  import de.weigend.s202.graph.TarjanSCCFinder;
7  import de.weigend.s202.reader.DependencyModel;
8
9  import java.util.ArrayList;
10 import java.util.HashMap;
11 import java.util.HashSet;
12 import java.util.LinkedHashSet;
13 import java.util.List;
14 import java.util.Map;
15 import java.util.Set;
16
17 /**
18  * Assigns a per-parent layer position ({@code localLevel}) to every
19  * element in a {@link DomainModel}. Each parent package is treated as a
20  * self-contained scope: only class dependencies whose source <em>and</em>
21  * target both live in the parent's subtree contribute to the sibling-only
22  * weighted graph. Refs that leave the parent are ignored -- they belong to
23  * the global {@code architectureLevel} via the package hierarchy.
24  *
25  * <p>Within each parent's sibling graph:
26  * <ol>
27  *   <li>Tarjan finds SCCs.</li>
28  *   <li>SCCs of size &gt; 1 are broken one edge at a time. The default
29  *       strategy evaluates every candidate edge in the whole SCC and cuts
30  *       the edge whose removal decomposes the cyclic component the most.
```

```
 *       Package-architecture contradictions, edge weight, and rank are used
 *       only as tie-breakers.</li>
 *   <li>Longest-path on the SCC-collapsed DAG assigns a {@code layer}
 *       (= local layer index) per sibling. Layer 0 = bottom of the box,
 *       higher values = visually higher.</li>
 * </ol>
 *
 * <p>Single-child containers stay at layer 0 -- no graph, no work.
 *
 * <p>The algorithm intentionally mirrors the structure of
 * {@link LevelCalculator}'s package-level computation, but applied
 * locally and using a different graph. Unlike package-level computation,
 * local SCC breaking is layout-oriented: it aims to stabilize the visible
 * architecture by reducing cyclic local structure, not to minimize the
 * number of visible upward edges. Local cycles are not surfaced as tangles;
 * the user-visible tangle list remains the global one.
 */
public class LocalLevelCalculator {

    private static final double RANK_THRESHOLD = 0.1;
    private final BreakMode breakMode;

    public LocalLevelCalculator() {
        this(BreakMode.MAX_CYCLE_BREAK);
    }

    LocalLevelCalculator(BreakMode breakMode) {
        this.breakMode = breakMode;
    }

    enum BreakMode {
        /**
         * Legacy local heuristic: remove every edge whose source has lower
         * weighted rank than its target by more than {@link #RANK_THRESHOLD}.
         */
        RANK,
        /**
         * Default local heuristic: inspect the whole SCC and remove one edge
         * whose cut maximally decomposes the cyclic component.
         */
        MAX_CYCLE_BREAK
    }

    public void assign(DomainModel domain, DependencyModel rawModel) {
        Map<String, List<CalculatedElementInfo>> childrenByParent = groupChildrenByParent(domain);
        for (Map.Entry<String, List<CalculatedElementInfo>> entry : childrenByParent.entrySet()) {
            assignForParent(entry.getValue(), domain, rawModel);
        }
    }

    private void assignForParent(List<CalculatedElementInfo> siblings,
                                 DomainModel domain, DependencyModel rawModel) {
        if (siblings.size() <= 1) {
            return; // single child stays at layer 0
        }

        Set<String> siblingFqns = new LinkedHashSet<>();
        for (CalculatedElementInfo c : siblings) {
            siblingFqns.add(c.fullName);
        }

        Map<String, Map<String, Integer>> weights = buildSiblingGraph(siblingFqns, domain, rawModel);
```

```
93          Map<String, Integer> layers = computeLayers(weights, siblings, siblingFqns);
94          for (CalculatedElementInfo s : siblings) {
95              s.setLocalLayerIndex(layers.getOrDefault(s.fullName, 0));
96          }
97      }
98
99      /**
100       * Build the weighted sibling-only edge map. For every class C in any
101       * sibling's subtree, every dep into another sibling's subtree
102       * contributes {@code callCount(C, dep)} weight; deps that leave the
103       * parent's subtree or stay inside the same sibling are skipped.
104       */
105     private Map<String, Map<String, Integer>> buildSiblingGraph(Set<String> siblingFqns,
106                                                                 DomainModel domain,
107                                                                 DependencyModel rawModel) {
108         Map<String, Map<String, Integer>> weights = new HashMap<>();
109         for (String s : siblingFqns) {
110             weights.put(s, new HashMap<>());
111         }
112         for (CalculatedElementInfo cls : domain.getAllClasses().values()) {
113             String fromSibling = containingSibling(cls.fullName, siblingFqns);
114             if (fromSibling == null) {
115                 continue;
116             }
117             for (String dep : cls.dependencies) {
118                 if (domain.getClass(dep) == null) {
119                     continue; // external -- not in any sibling
120                 }
121                 String toSibling = containingSibling(dep, siblingFqns);
122                 if (toSibling == null || toSibling.equals(fromSibling)) {
123                     continue; // out of parent OR intra-sibling
124                 }
125                 int w = callCount(cls.fullName, dep, rawModel);
126                 weights.get(fromSibling).merge(toSibling, w, Integer::sum);
127             }
128         }
129         return weights;
130     }
131
132     /**
133       * Walk up the fqn package chain until the first ancestor that is one
134       * of the siblings, or return {@code null} when none of the ancestors
135       * is -- the class lives outside the parent's subtree.
136       */
137     private static String containingSibling(String fqn, Set<String> siblingFqns) {
138         String current = fqn;
139         while (current != null && !current.isEmpty()) {
140             if (siblingFqns.contains(current)) {
141                 return current;
142             }
143             int dot = current.lastIndexOf('.');
144             if (dot < 0) {
145                 return null;
146             }
147             current = current.substring(0, dot);
148         }
149         return null;
150     }
151
152     private static Map<String, List<CalculatedElementInfo>> groupChildrenByParent(DomainModel domain) {
153         Map<String, List<CalculatedElementInfo>> result = new HashMap<>();
154         for (CalculatedElementInfo cls : domain.getAllClasses().values()) {
```

```
            result.computeIfAbsent(parentOf(cls.fullName), k -> new ArrayList<>()).add(cls);
        }
        for (CalculatedElementInfo pkg : domain.getAllPackages().values()) {
            result.computeIfAbsent(parentOf(pkg.fullName), k -> new ArrayList<>()).add(pkg);
        }
        return result;
    }

    private static String parentOf(String fqn) {
        int dot = fqn.lastIndexOf('.');
        return dot < 0 ? "" : fqn.substring(0, dot);
    }

    /**
      * Method-call count from class {@code from} to class {@code to}.
      * Falls back to 1 for structural deps (extends/implements/field type)
      * where no method-call info exists -- same convention the global
      * weighted package graph uses.
      */
    private static int callCount(String from, String to, DependencyModel rawModel) {
        DependencyModel.ClassInfo cls = rawModel.getClass(from);
        if (cls == null) {
            return 1;
        }
        int count = 0;
        String prefix = to + ".";
        for (DependencyModel.MethodInfo method : cls.methods.values()) {
            for (Map.Entry<String, Integer> call : method.methodCalls.entrySet()) {
                if (call.getKey().startsWith(prefix)) {
                    count += call.getValue();
                }
            }
        }
        return count > 0 ? count : 1;
    }

    // ----------- SCC-collapsed longest path with configurable SCC break -----------

    private Map<String, Integer> computeLayers(Map<String, Map<String, Integer>> weights,
                                               List<CalculatedElementInfo> siblings,
                                               Set<String> nodes) {
        Map<String, Set<String>> graph = new HashMap<>();
        for (String n : nodes) {
            graph.put(n, new HashSet<>(weights.getOrDefault(n, Map.of()).keySet()));
        }
        Map<String, CalculatedElementInfo> siblingByName = new HashMap<>();
        for (CalculatedElementInfo sibling : siblings) {
            siblingByName.put(sibling.fullName, sibling);
        }

        // Iteratively break local SCCs, then assign layers on the remaining DAG.
        // MAX_CYCLE_BREAK is the default; RANK remains available as a legacy
        // comparison mode for tests and experiments.
        boolean changed = true;
        while (changed) {
            changed = false;
            for (StronglyConnectedComponent scc : new TarjanSCCFinder(graph).findSCCs()) {
                if (scc.getSize() < 2) {
                    continue;
                }
                Set<String> members = scc.getMembers();
                Map<String, Double> rank = new HashMap<>();
```

```
            for (String m : members) {
                int out = 0;
                int in = 0;
                for (String other : members) {
                    if (other.equals(m)) {
                        continue;
                    }
                    out += weights.getOrDefault(m, Map.of()).getOrDefault(other, 0);
                    in  += weights.getOrDefault(other, Map.of()).getOrDefault(m, 0);
                }
                rank.put(m, (out - in) / (double) Math.max(1, out + in));
            }
            if (breakMode == BreakMode.MAX_CYCLE_BREAK) {
                EdgeCutCandidate best = bestCycleBreakingEdge(graph, weights, members, rank, siblingByName);
                if (best != null) {
                    graph.get(best.from()).remove(best.to());
                    changed = true;
                }
                continue;
            }
            // Legacy path: remove all rank-based back-edges found in this SCC.
            for (String from : new ArrayList<>(members)) {
                for (String to : new ArrayList<>(graph.getOrDefault(from, Set.of()))) {
                    if (!members.contains(to)) {
                        continue;
                    }
                    if (rank.get(from) < rank.get(to) - RANK_THRESHOLD) {
                        graph.get(from).remove(to);
                        changed = true;
                    }
                }
            }
        }
    }

    // SCC-collapsed longest-path level assignment.
    List<StronglyConnectedComponent> sccs = new TarjanSCCFinder(graph).findSCCs();
    Map<String, StronglyConnectedComponent> nodeToScc = new HashMap<>();
    for (StronglyConnectedComponent scc : sccs) {
        for (String m : scc.getMembers()) {
            nodeToScc.put(m, scc);
        }
    }
    Map<Integer, Set<Integer>> sccDeps = new HashMap<>();
    for (StronglyConnectedComponent scc : sccs) {
        sccDeps.put(scc.getId(), new HashSet<>());
        for (String m : scc.getMembers()) {
            for (String to : graph.getOrDefault(m, Set.of())) {
                StronglyConnectedComponent toScc = nodeToScc.get(to);
                if (toScc != null && toScc.getId() != scc.getId()) {
                    sccDeps.get(scc.getId()).add(toScc.getId());
                }
            }
        }
    }
    Map<Integer, Integer> sccLayers = new HashMap<>();
    for (StronglyConnectedComponent scc : sccs) {
        sccLayers.put(scc.getId(), 0);
    }
    boolean lvlChanged = true;
    while (lvlChanged) {
        lvlChanged = false;
```

```
            for (StronglyConnectedComponent scc : sccs) {
                int maxDep = -1;
                for (int depId : sccDeps.get(scc.getId())) {
                    maxDep = Math.max(maxDep, sccLayers.get(depId));
                }
                int newLayer = maxDep >= 0 ? maxDep + 1 : 0;
                if (sccLayers.get(scc.getId()) != newLayer) {
                    sccLayers.put(scc.getId(), newLayer);
                    lvlChanged = true;
                }
            }
        }

        Map<String, Integer> result = new HashMap<>();
        for (StronglyConnectedComponent scc : sccs) {
            int layer = sccLayers.get(scc.getId());
            for (String m : scc.getMembers()) {
                result.put(m, layer);
            }
        }
        return result;
    }

    private static EdgeCutCandidate bestCycleBreakingEdge(Map<String, Set<String>> graph,
                                                          Map<String, Map<String, Integer>> weights,
                                                          Set<String> members,
                                                          Map<String, Double> rank,
                                                          Map<String, CalculatedElementInfo> siblingByName) {
        EdgeCutCandidate best = null;
        for (String from : members) {
            for (String to : graph.getOrDefault(from, Set.of())) {
                if (!members.contains(to)) {
                    continue;
                }
                EdgeCutCandidate candidate = new EdgeCutCandidate(
                        from,
                        to,
                        cycleBreakScore(graph, members, from, to),
                        architectureContradiction(from, to, siblingByName),
                        weights.getOrDefault(from, Map.of()).getOrDefault(to, 0),
                        rank.getOrDefault(to, 0.0) - rank.getOrDefault(from, 0.0));
                if (best == null || candidate.compareTo(best) > 0) {
                    best = candidate;
                }
            }
        }
        return best;
    }

    /**
      * Score how much one candidate cut decomposes the current SCC. The score
      * compares the original SCC size squared with the sum of squared SCC sizes
      * after removing the candidate edge. A cut that splits one large cycle into
      * several smaller components therefore wins over a cut that leaves the SCC
      * almost intact.
      */
    private static int cycleBreakScore(Map<String, Set<String>> graph,
                                       Set<String> members,
                                       String cutFrom,
                                       String cutTo) {
        Map<String, Set<String>> copy = new HashMap<>();
        for (String member : members) {
```

```
341             copy.put(member, new HashSet<>());
342             for (String to : graph.getOrDefault(member, Set.of())) {
343                 if (members.contains(to) && !(member.equals(cutFrom) && to.equals(cutTo))) {
344                     copy.get(member).add(to);
345                 }
346             }
347         }
348 
349         int after = 0;
350         for (StronglyConnectedComponent scc : new TarjanSCCFinder(copy).findSCCs()) {
351             after += scc.getSize() * scc.getSize();
352         }
353         return members.size() * members.size() - after;
354     }
355 
356     /**
357       * Tie-breaker for package siblings: if an edge runs against the already
358       * computed package architectureLevel, prefer cutting it over an otherwise
359       * equivalent package edge. Class edges return 0 because classes are placed
360       * inside the package frame, not used to redefine that frame.
361       */
362     private static int architectureContradiction(String from,
363                                                  String to,
364                                                  Map<String, CalculatedElementInfo> siblingByName) {
365         CalculatedElementInfo fromInfo = siblingByName.get(from);
366         CalculatedElementInfo toInfo = siblingByName.get(to);
367         if (fromInfo == null || toInfo == null) {
368             return 0;
369         }
370         if (!"PACKAGE".equals(fromInfo.type) || !"PACKAGE".equals(toInfo.type)) {
371             return 0;
372         }
373         return fromInfo.architectureLevel < toInfo.architectureLevel ? 1 : 0;
374     }
375 
376     private record EdgeCutCandidate(String from,
377                                     String to,
378                                     int cycleBreakScore,
379                                     int architectureContradiction,
380                                     int weight,
381                                     double rankBackEdgeScore)
382             implements Comparable<EdgeCutCandidate> {
383         /**
384           * Higher is better. Order of preference:
385           * 1. decompose the local SCC as much as possible,
386           * 2. prefer cuts that preserve the package architecture hypothesis,
387           * 3. cut the weaker edge,
388           * 4. fall back to the legacy rank signal.
389           */
390         @Override
391         public int compareTo(EdgeCutCandidate other) {
392             int cmp = Integer.compare(cycleBreakScore, other.cycleBreakScore);
393             if (cmp != 0) {
394                 return cmp;
395             }
396             cmp = Integer.compare(architectureContradiction, other.architectureContradiction);
397             if (cmp != 0) {
398                 return cmp;
399             }
400             cmp = Integer.compare(other.weight, weight);
401             if (cmp != 0) {
402                 return cmp;
```

```
403                 }
404                 return Double.compare(rankBackEdgeScore, other.rankBackEdgeScore);
405             }
406         }
407 }
```

Appendix C – Questions and Answers

Why does S202 need a separate hierarchy for packages?

Because S202 orders classes and packages by different criteria. A class level is the depth of a node in the class dependency DAG after SCC handling: if class A depends on class B, then A belongs above B unless the edge is an explicit heuristic back-edge or both classes remain in the same SCC.

A package level answers a different question. Packages are containers, and a single package can contain many classes with dependencies in both directions. S202 therefore cannot derive package order from the deepest class inside the package. Instead, it compares the weighted relations between packages: when the traffic from package P_A to package P_B dominates the reverse traffic, P_A becomes the higher dependent package and P_B the lower package it depends on. S202 keeps equal or near-equal bidirectional traffic as a cyclic peer relationship.

This is why S202 computes class architecture levels and package architecture levels on separate graphs. The separation prevents class depth from accidentally lifting containers and prevents package aggregation from changing the meaning of class-level dependency depth.

It also prevents a feedback loop. If S202 derived package levels from class levels and then fed them back into class or layout ordering, each update could change the input of the next update. A package moves because one contained class moves; that package movement then changes sibling order or parent order; the changed order can expose another cycle or another dominant relation. Sorting and leveling algorithms then no longer operate on a stable graph. The result can depend on traversal order, classpath order, or on how often the loop is run. The system may converge accidentally, oscillate, or settle on a layout that is only an artifact of the update sequence. Separating the graphs makes each phase operate on fixed input and gives the invariants a well-defined result to check.

Why does S202 need both an architecture level and a local UI level?

Because these two numbers answer different questions. The architecture level of a class says how deep the class is in the global dependency graph: how many dependency steps separate it from the lower end of the class DAG after SCC handling. This is a system-wide analysis value.

The local level answers a layout question: where should this class or sub-package be placed relative to the other direct children of the same package? That question cannot be answered reliably from the global architecture level alone. A class can have a high global architecture level because it depends on deep chains elsewhere in the system, even though inside its own package it is not the highest local element. Conversely, a package-level coordinator may need to sit above a sub-package it calls into, even if their global architecture levels do not express that local relationship.

Using only the global DAG level for layout therefore over-constrains the picture. Elements can be pulled too high by dependencies outside their local container, and subsequent sorting passes try to repair a layout decision with the wrong input value. That is hard to make stable because the global level and the local sibling order are coupled through unrelated dependencies.

S202 keeps both values. The architecture level remains the semantic dependency depth used for global checks and reports. The local level is recomputed inside each package from the sibling-only graph and is used for visual ordering inside that package. The renderer can then place elements correctly within a container without changing what their global architecture level means.